

1. Introduction

Project Title:

StockEZ – MERN Stack Stock Trading Platform

2. Project Overview

Purpose:

StockEZ is a full-stack stock trading application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The main goal of the application is to provide users with a secure and interactive platform for monitoring stock market trends, viewing real-time stock information, managing investment portfolios, and performing stock buy and sell transactions. The application also provides an admin dashboard for managing users, stocks, transactions, and analytics.

Features:

- User registration and login system.
- JWT-based authentication and authorization.
- Home dashboard with market overview.
- Trending stocks and market summary.
- Stock listing with sorting, filtering, and search functionality.
- Interactive stock detail page with Chart.js graphs.
- Buy and sell stock functionality.
- Portfolio management with profit/loss tracking.
- Transaction history management.
- Admin dashboard for managing users, stocks, and transactions.
- Analytics dashboard with investment insights.
- RESTful API-based communication between frontend and backend.

- Responsive glassmorphism UI with modern dark theme.
-

3. Architecture

Frontend:

The frontend is developed using React.js and provides a modern, responsive, and interactive user interface. It consists of reusable components and pages such as Home Dashboard, Stock Market, Stock Details, Portfolio, Transactions, Authentication, User Profile, and Admin Dashboard. React Router DOM is used for navigation, Axios is used for API integration, and Chart.js is used to display interactive stock charts and financial data visualization.

Backend:

The backend is developed using Node.js and Express.js following the MVC (Model-View-Controller) architecture. It handles API requests, authentication, stock management, portfolio calculations, transaction processing, analytics, and administrative functionalities. Routes, controllers, middleware, and models are organized separately for better scalability and maintainability.

Database:

MongoDB is used as the database and Mongoose is used for schema modeling. The application contains collections for:

- Users – Stores user details and authentication information.
 - Stocks – Stores stock market information.
 - Portfolios – Stores user investment portfolios.
 - Transactions – Stores buy and sell transaction details.
 - Admin – Stores analytics and administrative data.
-

4. Setup Instructions

Prerequisites:

Install the following software before running the project:

- Node.js (v16 or above)
- npm (v8 or above)
- MongoDB or MongoDB Atlas
- Visual Studio Code
- Git and GitHub
- Postman for API testing

Installation:

1. Clone the repository:

```
git clone <repository-url>
```

2. Navigate to the project folder:

```
cd StockEZ
```

3. Install frontend dependencies:

```
cd client
```

```
npm install
```

4. Install backend dependencies:

```
cd server
```

```
npm install
```

5. Configure environment variables:

Create a .env file inside the server folder:

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_secret_key
```

5. Folder Structure

Client (React Frontend):

client/

├─ src/

| └─ components/

| └─ pages/

| └─ charts/

| └─ services/

| └─ context/

| └─ App.jsx

| └─ main.jsx

└─ package.json

Server (Node.js Backend):

server/

├─ models/

├─ controllers/

├─ routes/

├─ middleware/

├─ config/

└─ index.js

└─ package.json

6. Running the Application

Start Frontend:

Navigate to the client directory:

```
npm run dev
```

Frontend runs on:

<http://localhost:3001>

Start Backend:

Navigate to the server directory:

```
nodemon index.js
```

Backend runs on:

<http://localhost:3001>

7. API Documentation

Authentication APIs

Endpoint	Method	Description
/api/auth/register	POST	Register a new user
/api/auth/login	POST	Authenticate user
/api/auth/profile	GET	Get user profile

Stock APIs

Endpoint	Method	Description
/api/stocks	GET	Fetch all stocks

Endpoint	Method	Description
/api/stocks/	GET	Fetch stock details
/api/stocks	POST	Add a new stock (Admin)

Portfolio APIs

Endpoint	Method	Description
/api/portfolio	GET	View user portfolio
/api/portfolio/buy	POST	Buy stocks
/api/portfolio/sell	POST	Sell stocks

Transaction APIs

Endpoint	Method	Description
/api/transactions	POST	Create a new transaction
/api/transactions	GET	View transaction history

Admin APIs

Endpoint	Method	Description
/api/admin/users	GET	View all users
/api/admin/stocks	GET	Manage stocks
/api/admin/analytics	GET	View dashboard analytics

Example Response:

```
{  
  
  "success": true,  
  
  "message": "Operation completed successfully"  
}
```

8. Authentication

Authentication is handled using JWT (JSON Web Tokens). During login, the server verifies the user's credentials and generates a secure authentication token used for accessing protected resources. Passwords are encrypted using bcryptjs before being stored in the database. Authorization middleware ensures that only authenticated users can access protected routes, while role-based authorization allows only administrators to manage users, stocks, and analytics.

9. User Interface

The application contains the following UI pages:

- Landing Page
- Home Dashboard
- Stock Market Page
- Stock Detail Page
- Registration and Login Page
- Portfolio Page
- Transactions Page
- User Profile Page
- Admin Dashboard
- Stock Management Page
- User Management Page
- Analytics Dashboard

Screenshots can be attached in this section to showcase the interface.

10. Testing

The application is tested using:

- Postman for API testing.
- Manual testing for frontend components and user workflows.
- Database testing to verify CRUD operations.
- Authentication and authorization testing for secured routes.
- Portfolio calculation and transaction testing.

11. Known Issues

- Real-time stock prices may depend on external APIs and can experience delays.
- Historical stock data availability depends on third-party data providers.
- Large datasets may require pagination and additional optimization.
- Advanced technical indicators are not included in the current version.

12. Future Enhancements

- Integration with live stock market APIs.
- AI-based stock recommendations.
- Watchlist functionality.
- Advanced stock filtering and searching.
- Technical analysis indicators.
- Real-time notifications and price alerts.
- News integration for listed companies.
- Email notifications for transactions.
- Export portfolio reports.
- Improved analytics dashboard.
- Enhanced security and performance optimizations.